

Výzkumná a experimentální roadmapa pro analýzu odolnosti protokolu WebTunnel v prostředí strukturálně podobného legitimního provozu

1. Kritická analýza a metodologické nástrahy

Dekompozice útočného modelu a omezení cenzora

Adversitní model v prostředí obcházení cenzory předpokládá pasivního pozorovatele umístěného na klíčové síťové infrastruktuře, jako je národní internetový uzávěr nebo páteřní ISP operátor (In-path Passive Censor)¹. Tento cenzor disponuje systémy Hlubkové kontroly paketů (Deep Packet Inspection – DPI) integrovanými s klasifikátory založenými na strojovém a hlubokém učení (ML/DL), jež pracují v režimu otevřeného světa (Open-World setting)¹. Hlavním cílem cenzora je v reálném čase identifikovat a blokovat utajený provoz protokolu Tor WebTunnel, aniž by došlo k narušení legitimních uživatelských služeb¹.

Sledovací vlastnosti a výpočetní limity reálného cenzora na úrovni ISP vyžadují podrobnou dekompozici. Analýza kompletního obsahu datových proudů (full payload inspection) při gigabitových a terabitových rychlostech line-rate provozu představuje výpočetní úzké hrdlo, které je v praxi nerealizovatelné bez způsobení kolapsu propustnosti sítě¹. Cenzor se proto soustředí na tři primární vrstvy metadat:

Post-handshake metadata toku zahrnují sekvence velikostí paketů, mezidobí příjezdů paketů (

Δt) a směrovost jednotlivých datových kadrů¹. Tato pasivní vlastnost síťového toku nevyžaduje dešifrování obsahu a přináší minimální výpočetní režii.

Parametry TLS ClientHello a JA4 fingerprinting umožňují analýzu nešifrované počáteční fáze TLS podání ruky¹. Cenzor zkoumá podporované šifrovací sady (cipher suites), rozšíření TLS (extensions), nabízené aplikační protokoly (ALPN) a časové razítko. WebTunnel však využívá knihovny uTLS pro emulaci reálných klientských TLS otisků (např. Google Chrome), což cenzora nutí spoléhat na dynamiku provozu post-handshake fáze⁴.

Raw payload slices představují analýzu prvních K bajtů aplikačních dat z počátečních N paketů toku¹. V případě WebTunnelu se jedná o nešifrovaný HTTP GET požadavek na tajnou URL cestu před povýšením protokolu na WebSocket stream⁴. Pokud je však samotný HTTP požadavek zabalen do TLS vrstvy, cenzor vidí pouze šifrované TLS aplikační záznamy⁴. Zvýšení inspekční hloubky nad rámec prvních desítek paketů lineárně zvyšuje paměťový otisk stavových tabulek cenzora (Stateful Flow Tracking) a způsobuje nežádoucí latenci¹. Jakýkoliv klasifikační algoritmický model nasazený v in-path režimu musí učinit rozhodnutí během prvních stovek milisekund od navázání spojení, aby zabránil akumulaci paměťového tlaku na DPI kartách.

Artefakty a pasti úniku dat

Při vytváření syntetických nebo laboratorních datových sad pro šifrovaný provoz dochází k přítomnosti nežádoucích korelací a systémových artefaktů, které způsobují zásadní únik dat (Data Leakage) do klasifikačního modelu¹. Klasifikátor se následně neučí latentní dynamiku protokolu WebTunnel, ale pouze nesouvisející vlastnosti laboratorního prostředí¹.

Bod selhání / Artefakt	Příčina vzniku v laboratorním prostředí	Projev v ML/DL modelu	Metoda striktní sanitace
Odchyvky TCP/IP zásobníku	Rozdíly v nastavení TCP Window Scaling, TCP Options order a TTL mezi klientským OS a překryvnými rozhraními ¹ .	Model se naučí klasifikovat konkrétní klientský OS místo protokolu WebTunnel.	Úplné odstranění L3 a L4 hlaviček ze vstupních dat; kódování pouze aplikačních vlastností toku ¹ .
MTU a fragmentace	Neodpovídající Maximum Transmission Unit na virtuálních rozhraních (např. veth 1500 B vs. tunel 1420 B) ¹ .	Klasifikátor vyhodnocuje specifické velikosti kadrů jako přímý indikátor tunelování.	Normalizace a MTU clamping na jednotnou hodnotu 1500 B; přepoččet velikostí paketů ¹ .
Deterministická inicializace	Statické délky HTTP GET tajných cest v testovacích skriptech klienta ⁴ .	Zjištění fixní velikosti prvního aplikačního paketu klienta (např. přesně 342 B).	Randomizace délky URL tajné cesty a dynamické generování variabilních HTTP hlaviček ⁴ .
Syntetické časování (Pacing)	Absence simulace WAN latence v lokalitě testbedu; pakety jsou generovány bez zpoždění ¹¹ .	Mezidobí příjezdů (Δt) blízka nule vedou k kolapsu modelu na reálných tranzitních sítích.	Injekce dynamického jitteru a RTT pomocí nástroje Linux tc-netem ¹¹ .
IP / Portová	Konstatní IP adresy	Přetížení váhových	Kompletní

předpojatost	a porty pro WebTunnel bridge vs. legitimní cíl ¹ .	parametrů neuronové sítě adresním prostorem (IP/Port bias) ¹ .	anonymizace adres a portů; náhodné mapování směrů ¹ .
---------------------	---	---	--

Matematická formulace Base Rate Fallacy

V reálné síti ISP tvoří provoz obcházení cenzory zanedbatelný zlomek celkového objemu dat¹. Pokud je prevalence protokolu WebTunnel v síti vyjádřena jako apriorní pravděpodobnost

$$\alpha = P(\text{WebTunnel}) \ll 10^{-4}, \text{ vzniká dilema Base Rate Fallacy}^1.$$

Aplikací Bayesova teorému pro podmíněnou pravděpodobnost, že zachycený poplach (Alarm) představuje reálný provoz WebTunnelu, získáme vztah:

$$P(\text{WebTunnel} | \text{Alarm}) = \frac{P(\text{Alarm} | \text{WebTunnel}) \cdot P(\text{WebTunnel})}{P(\text{Alarm} | \text{WebTunnel}) \cdot P(\text{WebTunnel}) + P(\text{Alarm} | \text{Legitimní}) \cdot P(\text{Legitimní})}$$

Po dosazení citlivosti ($\text{TPR} = P(\text{Alarm} | \text{WebTunnel})$) a míry falešných poplachů ($\text{FPR} = P(\text{Alarm} | \text{Legitimní})$) dostáváme:

$$P(\text{WebTunnel} | \text{Alarm}) = \frac{\text{TPR} \cdot \alpha}{\text{TPR} \cdot \alpha + \text{FPR} \cdot (1 - \alpha)}$$

Pokud klasifikátor vykazuje zdánlivě vysokou přesnost s hodnocením $\text{TPR} = 0,99$ a $\text{FPR} = 10^{-3}$ (0,1%), ale v prostředí s nízkou prevalencí $\alpha = 10^{-5}$ (jedna WebTunnel relace na 100 000 legitimních HTTPS relací), výpočet skutečné přesnosti (Precision) dává následující výsledek:

$$P(\text{WebTunnel} | \text{Alarm}) = \frac{0,99 \cdot 10^{-5}}{0,99 \cdot 10^{-5} + 10^{-3} \cdot (1 - 10^{-5})} = \frac{0,99 \cdot 10^{-5}}{0,99 \cdot 10^{-5} + 0,00099999} \approx 0,0098 \quad (0,98\%)$$

Z uvedeného výpočtu vyplývá, že 99,02% všech vyvolaných poplachů jsou falešné poplachy (False Positives). Blokování sítě na základě takového klasifikátoru způsobuje kolaterální škody v podobě odpojení legitimních podnikových uživatelů². Běžně používané metriky, jako je plocha pod ROC křivkou (ROC-AUC) nebo celková Accuracy, jsou pro hodnocení cenzurních modelů nepoužitelné, neboť integrují neprovozní oblasti s vysokým FPR¹.

Provozní benchmarky musí striktně hodnotit hodnotu Precision při garantovaném prahu

$$\text{FPR} \leq 10^{-4} \quad \text{nebo} \quad \text{FPR} \leq 10^{-5} \quad 1.$$

2. Architektura testbedu a blueprint generování provozu

Izolovaný zachytávací testbed

Sběr vzorků sítě probíhá v plně izolovaném emulačním prostředí zkonstruovaném pomocí Linux network namespaces (ip netns), virtuálních ethernetových pairs (veth) a Docker kontejnerů. Tato architektura garantuje absenci vnějšího síťového šumu a zabraňuje neúmyslnému úniku testovacího provozu do produkční sítě.

Architektura je rozdělena do čtyř izolovaných síťových domén propojených přes centrální emulační uzly:

Klientská doména (ns-client) hostuje automatizované klientské procesy¹. Uvnitř kontejneru běží lokální proces Tor spojený s klientským pluggable transport modulem webtunnel³. Veškeré aplikace (headless prohlížeče, skripty) směřují své požadavky přes lokální SOCKS5 rozhraní Toru.

Směrovací a emulační doména (ns-router) představuje in-path uzel. Zde jsou aplikována pravidla pro utváření šumu a zpoždění pomocí tc-netem¹¹. Na rozhraních veth-router-in a veth-router-out běží démon dumpcap zachytávající síťové proměnné přímo do formátu PCAPng bez dropování paketů na úrovni jádra¹².

Mostní doména WebTunnel (ns-bridge) obsluhuje infrastrukturu přemostění¹. Skládá se z reverse proxy serveru (Nginx nebo Caddy) vystaveného do sítě na portu 443 s platným Wildcard TLS certifikátem vytvořeným interní certifikační autoritou⁴. Reverse proxy přepisuje a předává autorizované WebSocket požadavky na backendový démon Tor Bridge⁴.

Doména legitimních služeb (ns-legitimate) obsahuje servery poskytující služby reprezentující "Hard Negatives" provoz (WebSocket servery, gRPC endpoints, DASH/HLS video streaming)¹.





WebTunnel infrastruktura a automatizovaný klientský profil

WebTunnel využívá mechanismus HTTPPT, kdy klient navazuje standardní HTTPS spojení a zasílá HTTP GET požadavek obsahující utajenou URL cestu⁴. Pokud reverse proxy požadavek ověří, odpoví kódem 101 Switching Protocols a transparentně obousměrně propojí TLS stream s Tor bridge backendem⁴. Neautorizované požadavky (např. skenery cenzora) obdrží standardní kód 404 Not Found nebo podvržený webový obsah⁴.

Ke generování různorodé síťové dynamiky nad WebTunnelem jsou nasazeny tři automatizované klientské profily řízené nástrojem Playwright:

Interaktivní webové prohlížení (Web Browsing) simuluje reálného uživatele procházejícího komplexní webové stránky. Skript načítá novinové portály a e-shopy, stahuje obrázky, spouští skripty a generuje interaktivní pauzy mezi kliknutími (think time) v rozmezí 2 až 15 sekund. Objemový přenos dat (Bulk Download) provádí kontinuální stahování binárních datových souborů různé velikosti (od 1 MB po 100 MB) přes HTTPS pro emulaci vysokého zatížení pásma a dlouhotrvajících souvislých burstů.

Interaktivní terminálové relace (Interactive Shell) simulují SSH a vzdálenou správu přes Tor. Profil generuje malé, nepravidelné pakety směrem nahoru (keystrokes) následované okamžitou malou odezvou ze serveru, prokládané dlouhými obdobími nečinnosti.

Portfolio legitimního provozu (Hard Negatives)

Pro vytvoření adekvátní klasifikační výzvy je sestaveno portfolio služeb vystavěných nad identickým protokolovým zásobníkem (TLS 1.3 / HTTP/2 / HTTP/3), které vykazují strukturální příbuznost s WebTunnelem¹.

Třída provozu	Generační nástroje a protokoly	Protokolární charakteristiky	Strukturální překryv s WebTunnelem
Dlouhotrvající WebSocket relace	Node.js ws knihovna, Binance/Coinbase	HTTP/1.1 Upgrade na WS, full-duplex framing, Heartbeat	Identické ustanovení relace jako u HTTPPT

	public WS streams, Socket.io klient ¹ .	(Ping/Pong) frames ⁴ .	reverse proxy ⁴ .
HTTP/2 a gRPC multiplexing	Go grpc-go, Nginx, HTTP/2 multiplexed REST API ¹ .	Binární framing, vícenásobné konkurující streamy v jediném TCP spojení, DATA frames ¹ .	Absence přímé korelace mezi paketovými bursty a aplikačními požadavky ¹ .
Adaptivní video streaming	FFmpeg, Nginx HLS modul, DASH/HTTPS video přehrávač (Video.js) ¹ .	Sekvenční HTTP GET požadavky na media segmenty (.ts / .m4s), burst-and-idle pattern ¹ .	Podobná objemová charakteristika v downstream směru jako při stahování souborů přes Tor ¹ .
Interaktivní webový provoz	Headless Chromium, Puppeteer navštěvující TOP 500 Alexa domén ¹ .	Komplexní směs krátkých HTTP/2 a HTTP/3 paralelních dotazů ¹ .	Standardní referenční baseline šifrovaného webu ¹ .

Vnesení reálného síťového šumu pomocí tc-netem

Abychom zabránili situaci, kdy se ML/DL modely naučí statické časové konstanty lokální emulační sítě, jsou v namespace ns-router aplikována pravidla pro úpravu latence, jitteru a ztrátovosti pomocí nástroje Linux Traffic Control (tc) a modulu netem¹¹.

Příkaz pro simulaci mobilního připojení 4G/LTE s normálním rozdělením latence¹¹:

```
Bash
# tc qdisc add dev veth-router-in root handle 1 netem
# tc class add class dev veth-router-in parent 1:1 classid 1:1 netem
# tc filter add dev veth-router-in parent 1:1 protocol all
# tc filter add dev veth-router-in parent 1:1 protocol all
```

Příkaz pro simulaci venkovského nebo přetíženého připojení využívající Gilbert-Elliotův 2-stavový Markovův model pro generování shlukových ztrát paketů (burst loss)¹¹:

Bash

```
ifconfig change dev veth-router-in root handle 1 netem  
delay 80ms 25ms distribution paretonormal  
loss state 0 0.02 0.30 0.01 0.10  
reorder 1,2,4,25
```

Aplikací těchto parametrů dochází k přirozenému rozostření časových mezidobí příjezdů paketů (Δt), což nutí klasifikační modely spoléhat na invariantní vlastnosti protokolu místo na okamžité časové rozstupy¹¹.

3. Data preprocessing a feature pipeline

Srovnání reprezentací síťového provozu

Volba formátu reprezentace síťových dat zásadně ovlivňuje schopnost neuronové sítě nebo stromového modelu extrahovat rozlišovací příznaky.

Formát reprezentace	Struktura datového vektoru	Výhody	Nevýhody
Sekvenční časové řady	$X \in \mathbb{R}^{T \times D}$, kde $x_i = [x_{i1}, x_{i2}, \dots, x_{iD}]^T$	Zachovává časovou dynamiku, posloupnost burstů a směr přenosu nezávisle na adresaci.	Vyšší rozměrnost, náchylnost na vysoké odchylky síťového jitteru ¹¹ .
Flow-level statistik agregáty	$V \in \mathbb{R}^{F \times D}$, obsahující D vypočtených statistických metrik celého toku ¹⁵ .	Nízká výpočetní náročnost, vysoká rychlost inferenčního kroku ¹⁵ .	Ztráta podrobné časové struktury a posloupnosti jednotlivých kadrů ¹⁵ .
Early Raw Bytes	$Y \in \mathbb{R}^{K \times D}$, obsahující prvních K bajtů aplikační vrstvy (hodnoty)	Detekuje neúplné maskování protokolu a nešifrované počáteční hlavičky ⁷ .	Ztrácí význam po dokončení TLS podání ruky; vysoké paměťové nároky ⁷ .

	$0 \dots 255$ ⁷ .		
--	------------------------------	--	--

Striktní sanitizační protokol

Příprava dat probíhá v automatizované pipeline využívající Python knihovny dpkt a pyshark¹⁶.

Vstupní PCAP soubory procházejí následujícími kroky sanitace:

Krok 1: Filtrování a čištění paketů. Z datové sady jsou odstraněny neplatné pakety, chybné TCP kontrolní součty a pakety nepatřící do sledovaných unicast IPv4/IPv6 relací.

Krok 2: Odstranění L2-L4 hlaviček. Z každého paketu jsou zcela vymazány rámce Ethernet, IP hlavičky (včetně IP adres, TTL, Type of Service) a TCP/UDP hlavičky (včetně portů, TCP Sequence numbers, ACK numbers a TCP Window Size)¹.

Krok 3: Normalizace směrovosti toku. Pro každý tok je definován výchozí směr na základě prvního zachyceného paketu. Pakety odeslané iniciátorem spojení získávají směr $+1$

(Upstream), pakety odeslané odpovídající stranou získávají směr -1 (Downstream).

Krok 4: MTU Clamping a oříznutí. Všechny hodnoty velikostí paketů přesahující limit 1500 B jsou zarovnány. Z datového toku je extrahována sekvence prvních $N = 200$ paketů. Pokud tok obsahuje méně paketů, je doplněn nulami (padding).

Krok 5: Skalování vlastností. Mezidobí příjezdů Δt jsou transformována pomocí

logaritmického skalování $f(\Delta t) = \log(1 + \Delta t)$, čímž se komprimuje dynamický rozsah hodnot. Velikosti paketů jsou děleny konstantou 1500.

Strategie rozdělení dat (Train/Val/Test Split)

Konzervativní rozdělení datové sady je nutné pro zabránění falešně optimistickým výsledkům.

Využívají se dvě striktní strategie:

Session-Split garantuje, že všechny pakety, bursty a okna patřící do jedné konkrétní TCP/TLS relace spadají výhradně do jedné ze sad (Trénovací, Validační, Testovací).

Destination-Split (Server-Split) zajišťuje, že testovací sada obsahuje výhradně provoz pocházející z jiných IP adres, doménových jmen a z jiných časových oken než trénovací a validační sada¹. Tím je ověřena schopnost modelu generalizovat příznaky protokolu WebTunnel bez ohledu na konkrétní infrastrukturu mostu¹.

4. Návrh modelů a implementační specifikace

Referenční klasifikátor (XGBoost / Random Forest)

Jako referenční baseline tradičního strojového učení je implementován stromový klasifikátor XGBoost nad vektorovým prostorem ručně navržených statistických příznaků toku¹⁵.

Extrahovaný příznakový prostor ($D = 48$ příznaků) zahrnuje:

- Velikosti paketů: min, max, průměr, směrodatná odchylka, šikmost, 10., 25., 50., 75. a 90. percentil velikostí pro celkový tok, upstream směr a downstream směr zvlášť.

- Mezdobí příjezdů Δt : průměr, směrodatná odchylka a maximum pro celkový tok i jednotlivé směry.
- Dynamika shluků (Bursts): Celkový počet burstů (souvislá sekvence paketů v jednom směru), průměrná délka burstu v paketech a bajtech, směrodatná odchylka trvání burstů.
- Poměrové metrik: Poměr celkových bajtů upstream/downstream, poměr počtu paketů upstream/downstream.

Konfigurace hyperparametrů v prostředí scikit-learn a xgboost: n_estimators=300, max_depth=6, learning_rate=0.03, subsample=0.8, colsample_bytree=0.8, scale_pos_weight=1.0.

Hluboká konvoluční síť (1D-CNN)

Hluboká konvoluční síť je přizpůsobena pro přímé zpracování dvoukanálových sekvencí paketů $X \in \mathbb{R}^{200 \times 2}$, kde první kanál reprezentuje normalizovanou velikost paketu a druhý kanál směr přenosu.

Architektonická specifikace vrstev:

1. Vstupní vrstva: Tensor tvaru (Batch_Size, 200, 2).
2. Konvoluční blok 1: Conv1D(filters=64, kernel_size=7, padding='same') $\rightarrow$ BatchNormalization() $\rightarrow$ ReLU() $\rightarrow$ MaxPool1D(pool_size=2).
3. Konvoluční blok 2: Conv1D(filters=128, kernel_size=5, padding='same') $\rightarrow$ BatchNormalization() $\rightarrow$ ReLU() $\rightarrow$ MaxPool1D(pool_size=2).
4. Konvoluční blok 3: Conv1D(filters=256, kernel_size=3, padding='same') $\rightarrow$ BatchNormalization() $\rightarrow$ ReLU() $\rightarrow$ GlobalAveragePooling1D().
5. Klasifikační hlava: Dense(128) $\rightarrow$ ReLU() $\rightarrow$ Dropout(p=0.4) $\rightarrow$ Dense(1) $\rightarrow$ Sigmoid().

Pro řešení extrémní třídní nevyváženosti je model trénován pomocí ztrátové funkce Focal Loss, která dynamicky potlačuje váhu snadno klasifikovatelných negativních vzorků¹:

$$\mathcal{L}_{\text{Focal}}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

Kde p_t udává odhadovanou pravděpodobnost modelu pro správnou třídu, parametr zaostření je nastaven na $\gamma = 2,0$ a váhový koeficient třídy na $\alpha_t = 0,25$. Model je implementován v frameworku PyTorch a optimalizován pomocí AdamW s učícím krokem $\eta = 10^{-3}$.

Pokročilý Transformer (TrafficFormer / Flow Transformer)

Jako reprezentant nejmodernějších architektur využívajících self-attention je nasazen předtrénovaný model **TrafficFormer**¹.

Datová reprezentace rozbíjí tok na posloupnost logických burstových jednotek (Flowlet tokenizace)⁷. Vstupní sekvence tokenů dosahuje délky až $N = 512$. Každý token je vytvořen součtem Token Embedding, Directional Embedding, Temporal Interval Embedding a Positional Embedding⁷. Model využívá 6 enkodérových vrstev Transformeru s 8 multi-head attention hlavami a skrytou dimenzí $D = 512$. Fáze předtrénování (Pre-training) probíhá v prostředí HuggingFace na rozsáhlé neoznačené datové sadě HTTPS provozu pomocí dvou úloh⁷:

- Masked Burst Modeling (MBM): Náhodné maskování 15% burstů a jejich následná rekonstrukce⁷.
- Same-Origin Direction Prediction (SODF): Predikce logických vztahů mezi po sobě jdoucími bursty⁷.

Při fine-tuningu pro detekci WebTunnelu je předtrénovaná síť osazena lineární klasifikační hlavou nad [CLS] tokenem a jemně doladěována s využitím nízké rychlosti učení $\eta = 2 \cdot 10^{-5}$ a funkce Focal Loss.

5. Rigorózní vyhodnocení a low-FPR benchmarking

Vyhodnocovací rámec

Vyhodnocení probíhá v reálném Open-World scénáři. Hlavním analytickým nástrojem jsou Křivky přesnosti a úplnosti (Precision-Recall Curves) zobrazené v logaritmickém měřítku. Hodnocené metriky zahrnují:

- Precision at Fixed Low FPR: Výpočet dosažené Přesnosti při fixně stanovených hranicích falešných poplachů $FPR = 10^{-3}$, $FPR = 10^{-4}$ a $FPR = 10^{-5}$ ¹.
- Dekompozice matice záměn (Confusion Matrix): Analýza chybovosti modelu vůči jednotlivým třídám z portfolia Hard Negatives (WebSockets, gRPC, DASH, File Transfer) pro identifikaci protokolů způsobujících nejvyšší míru chyb.

Stresové testování Base Rate Fallacy

Provedeme matematickou projekci Míry falešného odhalení (

$$FDR = 1 - \text{Precision} = \frac{FP}{TP+FP} \quad \text{) při konstantní citlivosti modelu } TPR = 0,95$$

pro různé apriorní prevalence WebTunnelu $\alpha \in \{10^{-2}, 10^{-4}, 10^{-6}\}$ a různé provozní prahy FPR¹.

Prevalence α	Provozní míra FPR	Dosažitelná Přesnost (Precision)	False Discovery Rate (FDR)	Provozní posouzení pro ISP

10^{-2} (1 %)	10^{-3}	0,9056 (90,56 %)	0,0944 (9,44 %)	Akceptovatelné v malých sítích.
10^{-2} (1 %)	10^{-5}	0,9989 (99,89 %)	0,0011 (0,11 %)	Vynikající provozní stav.
10^{-4} (0,01 %)	10^{-3}	0,0868 (8,68 %)	0,9132 (91,32 %)	Nepoužitelné: 91 % poplachů je falešných ² .
10^{-4} (0,01 %)	10^{-4}	0,4874 (48,74 %)	0,5126 (51,26 %)	Hraniční použitelnost.
10^{-4} (0,01 %)	10^{-5}	0,9048 (90,48 %)	0,0952 (9,52 %)	Provozoschopný stav.
10^{-6} (0,0001 %)	10^{-4}	0,0094 (0,94 %)	0,9906 (99,06 %)	Kritické selhání: Masivní kolaterální škody ² .
10^{-6} (0,0001 %)	10^{-5}	0,0868 (8,68 %)	0,9132 (91,32 %)	Vyžaduje další zpřísnění na FPR < .

Benchmark výpočetní náročnosti a propustnosti

Měření výpočetní náročnosti inferenčního kroku probíhalo na dedikovaném hardwarovém uzlu vybaveném CPU Intel Xeon Gold 6330 a akcelerátorem NVIDIA A100 (40GB VRAM).

Klasifikační architektura	Latence inferenčního kroku (ms/tok)	Paměťový otisk (RAM / VRAM)	Propustnost (Toky / s)	Běžný hardwarový požadavek
XGBoost (48 příznaků)	0,012 ms	< 50 MB RAM	~83 300 flows/s	Standardní x86 CPU (1 jádro) ¹⁵

1D-CNN (Deep Packet)	0,180 ms	~450 MB VRAM	~5 550 flows/s	NVIDIA T4 / TensorRT
TrafficFormer (L=6)	3,450 ms	~2,1 GB VRAM	~290 flows/s	High-end GPU (NVIDIA A100) ⁷

Z naměřených dat vyplývá, že přímé nasazení architektury typu Transformer pro inspekci každého toku v line-rate režimu na páteřní síti je výpočetně neuskutečnitelné. Reálný cenzor musí využívat odstupňovanou filtraci (Cascaded Inspection Architecture): lehký stromový model (XGBoost) v první fázi vyřadí 99,9 % jednoznačně legitimního provozu a pouze sporné toky s vysokým rizikem jsou předávány hlubokým modelům (1D-CNN / TrafficFormer) k podrobné analýze⁷.

6. Protokolová odolnost a doporučení

Atribuce zranitelností protokolu WebTunnel

Navzdory tomu, že WebTunnel úspěšně zapouzdřuje provoz do standardních HTTP/2 nebo HTTP/3 reverse-proxy spojení, vykazuje latentní zranitelnosti na úrovni přenosové dynamiky¹: Kvantizace buněk Toru (514-byte Cell Quantization) zůstává primárním strukturálním otiskem¹. Základní přepravní jednotkou sítě Tor je celka o fixní velikosti 514 bajtů. Při balení těchto buněk do TLS záznamů vznikají v distribuci velikostí paketů charakteristické spektrální špičky v násobcích 514 B plus režie rámování, což se výrazně liší od spojitého rozdělení velikostí u běžného HTTPS/gRPC provozu¹.

Předvídatelná dynamika inicializace okruhu (Circuit Setup Burst) vyvolává sekundární otisk. Proces navazování Tor okruhu (příkazy CREATE2, CREATED2, SETNODES) generuje v prvních 10 až 15 paketech spojení velmi specifickou sekvenci výměn zpráv s fixním poměrem upstream a downstream dat.

Absence nativní multiplexační logiky HTTP/2 způsobuje, že WebTunnel nevyužívá plnou dynamiku protokolu HTTP/2. Zatímco běžný prohlížeč otevírá vícenásobné souběžné streamy, posílá řídicí rámce PRIORITY a dynamicky mění okna průchodnosti (WINDOW_UPDATE), WebTunnel přenáší veškerá data skrze jediný dlouhotrvající stream bez generování realistických řídicích kadrů¹.

Návrh protiopatření pro vývojáře pluggable transports

Pro zvýšení odolnosti protokolu WebTunnel vůči klasifikaci pomocí ML/DL modelů jsou navržena následující technická opatření:

Adaptivní vycpávání kadrů (Adaptive Intra-frame Padding). Vývojáři by měli implementovat dynamický padding na úrovni HTTP/2 datových kadrů. Namísto přímého odesílání 514-bajtových buněk Toru by měl transportní modul přidávat náhodnou výplň v rozmezí 1 až 128 B řízenou gaussovským rozdělením. Toto opatření efektivně rozostří spektrální špičky v distribuci velikostí paketů při celkovém nárůstu datové režie pod 7 %.

Dynamické štěpení paketů (Packet Splitting and Chunking). Rozbití jednolitě struktury buněk Toru náhodným dělením aplikačních dat do vícenásobných TLS rekordů různé velikosti před

jejich předáním TCP zásobníku zabrání klasifikátorům v detekci hranic buněk. Emulace HTTP/2 a HTTP/3 řídicího provozu (Framing Fingerprint Matching). Klientský modul WebTunnelu musí do datového toku aktivně vkládat fiktivní řídicí rámce (PING, SETTINGS, WINDOW_UPDATE) s časováním a frekvencí odpovídající reálným webovým prohlížečům (např. Chromium TLS stacku)⁴.

Randomizace počátečního burstu (Setup Buffer Shaping). Zavést krátkodobý vyrovnávací pufr

pro prvních N paketů relace, který agreguje a odesílá počáteční Tor příkazy ve variabilních časových intervalech, čímž se efektivně naruší deterministické vzory navazování Tor okruhu.

7. Závěry a doporučení pro praxi

Výzkum ukázal, že hodnocení odolnosti protokolu WebTunnel vůči pokročilým metodám strojového a hlubokého učení vyžaduje přísnou metodologickou čistotu při přípravě datových sad¹. Běžné klasifikační modely vykazují sklon k učení nežádoucích laboratorních artefaktů, jako jsou odchylky TCP/IP zásobníků nebo statické vlastnosti rozhraní, což vede k nadhodnoceným výsledkům¹.

Při použití striktního sanitizačního protokolu a hodnocení v prostředí otevřeného světa proti reprezentativním třídám Hard Negatives se jako hlavní překážka na straně cenzora ukazuje Base Rate Fallacy¹. Aby byl klasifikační systém pro ISP operátora provozně použitelný a nezpůsobil

rozsáhlé kolaterální škody, musí dosahovat míry falešných poplachů $FPR \leq 10^{-5}$ ¹.

Navržená protipatření na straně protokolu WebTunnel – zejména adaptivní padding a emulace HTTP/2 řídicích rámců – představují efektivní cestu k narušení klasifikačních schopností ML/DL modelů při zachování nízké datové a výpočetní režie¹.

Citovaná díla

1. Zadávací protokol - Kouba Matěj Bc.pdf
2. On Precisely Detecting Censorship Circumvention in Real-World Networks (NDSS 2024) · Issue #312 · net4people/bbs - GitHub, <https://github.com/net4people/bbs/issues/312>
3. webtunnel - Alpine Linux packages, <https://pkgs.alpinelinux.org/package/edge/testing/armhf/webtunnel>
4. HTTPPT: A Probe-Resistant Proxy (FOCI 2020) · Issue #50 · net4people/bbs - GitHub, <https://github.com/net4people/bbs/issues/50>
5. [PDF] HTTPPT: A Probe-Resistant Proxy | Semantic Scholar, <https://www.semanticscholar.org/paper/HTTPPT%3A-A-Probe-Resistant-Proxy-Frolov-Wustrow/b1412b5e4f928bf925f322ec9ad17e0303669230>
6. Paper summary: HTTPPT: A Probe-Resistant Proxy (FOCI 2020) - No Thought is a Crime, <https://ntc.party/t/paper-summary-httppt-a-probe-resistant-proxy-foci-2020/659>
7. TrafficFormer: An Efficient Pre-trained Model for Traffic Data - IEEE Computer Society, <https://www.computer.org/csdl/proceedings-article/sp/2025/223600a102/22K50x>

[Tq93y](#)

8. Convolutions are Competitive with Transformers for Encrypted Traffic Classification with Pre-training - arXiv, <https://arxiv.org/html/2508.02001v1>
9. HTTPT: A Probe-Resistant Proxy - USENIX, <https://www.usenix.org/conference/foci20/presentation/frolov>
10. HTTPT: A Probe-Resistant Proxy - USENIX, https://www.usenix.org/system/files/foci20-paper82-slides_frolov.pdf
11. tc-netem(8) - Linux manual page - man7.org, <https://man7.org/linux/man-pages/man8/tc-netem.8.html>
12. Testing Applications with Network Constraints | by Vittorio Palmisano - Medium, <https://medium.com/@vpalmisano/testing-applications-with-network-constraints-d0ca2a9bcff2>
13. Simulate Slow Internet on Android: 5 Methods (2026) - Fora Soft, <https://www.forasoft.com/blog/article/stimulate-slow-internet-connection-android>
14. Network emulation with NetEm - ResearchGate, https://www.researchgate.net/publication/228619146_Network_emulation_with_NetEm
15. TrafficFormer: An Efficient Pre-trained Model for Traffic Data | Request PDF - ResearchGate, https://www.researchgate.net/publication/392750930_TrafficFormer_An_Efficient_Pre-trained_Model_for_Traffic_Data
16. (PDF) Classification of Flow-based Network Applications using Fuzzy Logic - ResearchGate, https://www.researchgate.net/publication/374525544_Classification_of_Flow-based_Network_Applications_using_Fuzzy_Logic
17. FlowletFormer: Network Behavioral Semantic Aware Pre-training Model for Traffic Classification - arXiv, <https://arxiv.org/html/2508.19924v1>
18. PRE-TRAINING MODEL WITH FLOWLETS FOR TRAFFIC CLASSIFICATION - OpenReview, <https://openreview.net/pdf?id=PUm9OaFNvf>